

- Es su responsabilidad cerciorarse de que toda entrega que realice de manera electrónica en la plataforma sea correcta.
- **No se aceptan tareas, practica ni programas por correo electrónico, estas se entregarán en clase o vía una plataforma educativa según se indique.**



Facultad de
Ingeniería

Estructuras de datos y algoritmos I

Profesora: Elba Karen Sáenz García

PROGRAMA DE ESTUDIO

ESTRUCTURA DE DATOS Y ALGORITMOS I		2	10
Asignatura		Semestre	Créditos
INGENIERÍA ELÉCTRICA	INGENIERÍA EN COMPUTACIÓN	INGENIERÍA EN COMPUTACIÓN	
División	Departamento	Licenciatura	
Asignatura:		Horas/semana:	
Obligatoria	☒	Teóricas	4.0
Optativa	☐	Prácticas	2.0
		Total	6.0
		Horas/semestre:	
		Teóricas	64.0
		Prácticas	32.0
		Total	96.0

Modalidad: Curso teórico-práctico

Seriación obligatoria antecedente: Fundamentos de Programación

Seriación obligatoria consecuente: Programación Orientada a Objetos, Estructura de Datos y Algoritmos

II

Objetivo(s) del curso:

- El alumno analizará problemas de almacenamiento, recuperación y ordenamiento de datos y algoritmos, utilizando las estructuras para representarlos en código y las técnicas de operación más eficientes.



Facultad de
Ingeniería

Tema 1

Estructuras de datos

Profesora: Elba Karen Sáenz García

Tema 1

- **1. Estructura de datos (34 horas)**

Objetivo: El alumno resolverá problemas de almacenamiento, recuperación y ordenamiento de datos y las técnicas de representación más eficientes, utilizando las estructuras para representarlos.

-

Contenido:

1.1. Representación de datos en memoria.

1.1.1. Tipos primitivos.

1.1.2. Arreglos.

1.1.3. Apuntadores.

1.1.4. Tipo de dato abstracto.

1.2. Administración del almacenamiento en tiempo de ejecución.

¿Qué es un dato?

- Puede referirse a un número, letra o cualquier símbolo que representa una palabra, cantidad o medida que se suministra a la computadora como entrada y la máquina almacena en un determinado formato.
- Son las características sobre las cuales opera un algoritmo

Características/atributos de los datos:

- Un nombre
- Un tipo
- Un valor

Tipo de dato:

- Indica a la computadora (y/o al programador) sobre la clase de datos que se va a trabajar.
- Impone restricciones en los datos, como qué valores pueden tomar y qué operaciones se pueden realizar.
- Los tipos de **datos comunes o primitivos** son:
 - Numéricos: Enteros, Reales
 - Caracteres
 - Alfanumérico
 - Lógicos

Tipos de datos primitivos en C

- **char** -> Caracteres
- **int** Números enteros
- **float** Números en coma flotante
- **double** Números en coma flotante de doble precisión
- **void** Tipo nulo
- Punteros -> Direcciones de memoria

Modificadores

- **Tamaño**
 - short (int por defecto)
 - long (int por defecto)
- **Signo: aplicable a char, short, int y long**
 - signed
 - unsigned

Enteros en C

Tipo de dato	Espacio en memoria	Valor Mínimo	Valor Máximo
char	8 bits	-128	127
unsigned char		0	255
short	16 bits	-32768	32767
unsigned short		0	65535
long	32 bits	-2147483648	2147483647
unsigned long		0	4294967295

Números Reales (Coma Flotante)

Tipo de dato	Espacio en memoria	Mínimo (valor absoluto)	Máximo (valor absoluto)	Dígitos significativos
float	32 bits	1.2×10^{-38}	3.4×10^{38}	6
double	64 bits	2.2×10^{-308}	1.8×10^{308}	15
long double	80 bits	3.4×10^{-4932}	1.2×10^{4932}	18

```
uint32_t = 0..4294967295
int32_t = -2147483648..2147483647
uint64_t = 0..18446744073709551615
int64_t = -9223372036854775808..9223372036854775807
size_t = 0..18446744073709551615
ssize_t = -9223372036854775808..9223372036854775807
pid_t = -2147483648..2147483647
time_t = -9223372036854775808..9223372036854775807
intptr_t = -9223372036854775808..9223372036854775807
unsigned char = 0..255
char = -128..127
uint8_t = 0..255
int8_t = -128..127
uint16_t = 0..65535
int16_t = -32768..32767
int = -2147483648..2147483647
long int = -9223372036854775808..9223372036854775807
long long int = -9223372036854775808..9223372036854775807
off_t = -9223372036854775808..9223372036854775807
```

Programa

```
tamato int 4  
tamato short 2  
tamato long 4  
tamato short int 2  
tamato long int 4
```

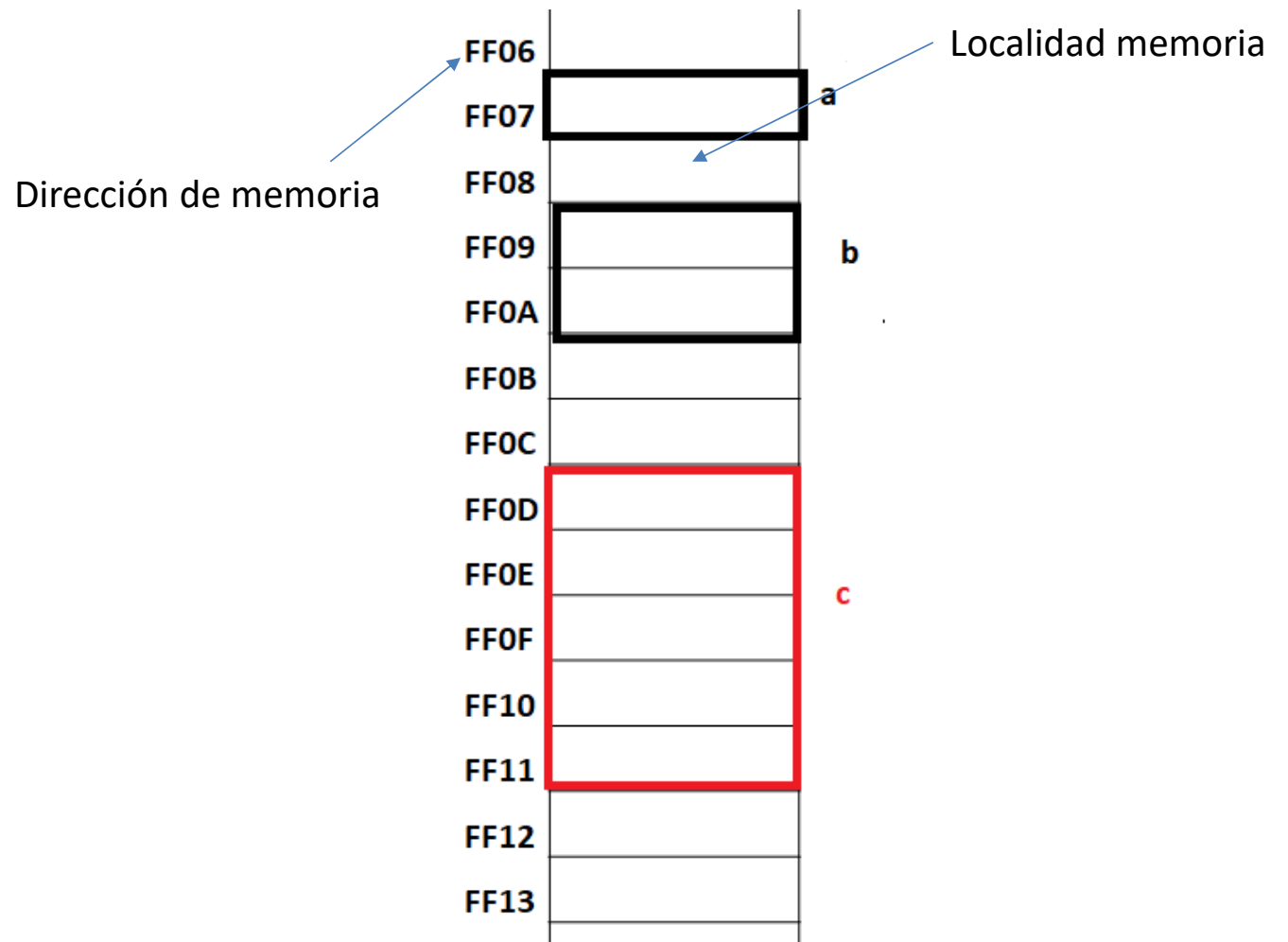
```
Tamato char 1:  
Tamato int 4:  
Tamato float 4:  
Tamato double 8:  
Tamato shortint 2:  
Tamato long 4:  
Tamato long double 16:
```

Representación en memoria

char a;

short int b;

float c;



Estructura de datos

- Es un conjunto de elementos de información que posee un **nombre** y está caracterizado por una organización y por las operaciones que se definen en ella.

Estructuras de datos fundamentales y estructurados

- Para procesar **información/datos**
 - Almacenarlos en la memoria computadora .
- Los datos a almacenar se organizan en:
 - Tipos datos simples (fundamentales)
 - Tipos datos estructurados
 - Con un nombre se hace referencia a un conjunto de datos (varias localidades de memoria)

Ejemplo arreglos

Las estructuras de datos pueden ser:

- Estáticas: El tamaño ocupado en memoria se define antes de que el programa se ejecute y no puede modificarse dicho tamaño durante la ejecución del programa.
- Dinámicas : No tienen las limitaciones o restricciones en el tamaño de memoria ocupada

Estructura de Datos

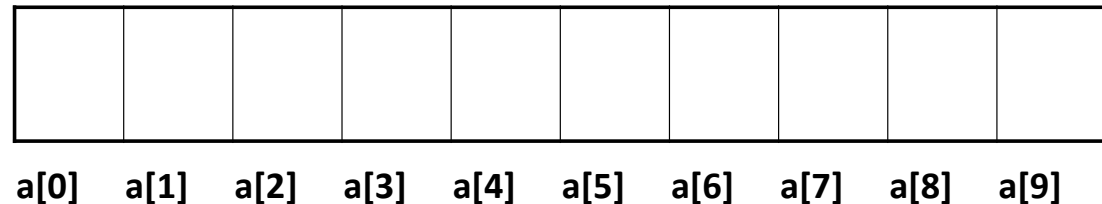
- Simples o primitivas: variables
- Compuestas: registros/estructuras, archivos, arreglos
- Lineales: Arreglos Listas Pilas Colas
- No lineales: Árboles y Grafos

Arreglo

- Colección finita , homogénea y ordenada de elementos.

Arreglos Unidimensionales (vectores)

- Partes
 - Nombre(identificador)
 - Los componentes(elementos)
 - Los índices
- Vista conceptual



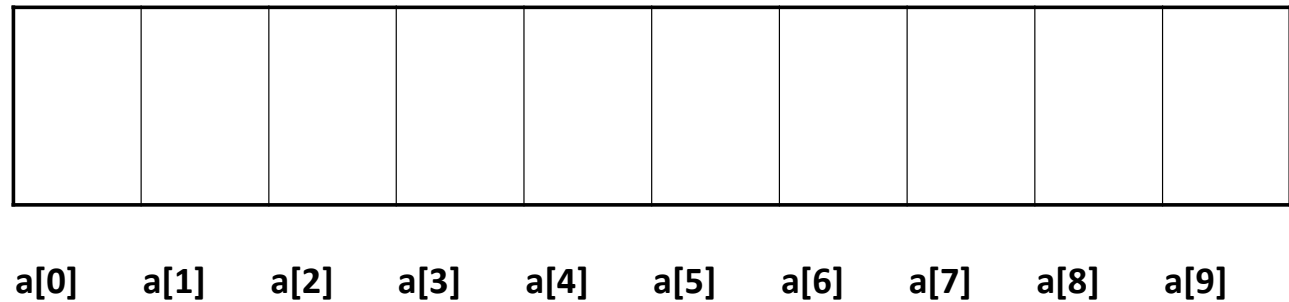
TipoEntero a[tamañoDim];

TipoEntero a[índicePrimerElemento,....., índiceÚltimoElemento];

Declaración de Arreglos unidimensionales en C (Estáticos o contiguos)

***tipoDato** nombre_arr [Tamdim1];*

int a[10];



Representación en memoria

short int a[7];

Elementos
acomodados en forma
consecutiva en
memoria

FF06		a[0]
FF07		
FF08		a[1]
FF09		
FF0A		a[2]
FF0B		
FF0C		a[3]
FF0D		
FF0E		a[4]
FF0F		
FF10		a[5]
FF11		
FF12		a[6]
FF13		
FF14		

Operaciones

- Leer /Escribir(mostrar)
- Asignación
- Modificar(actualización)

Otras, mediante algoritmos

- Ordenar
- Buscar

```
int a[10];
```

Asignación

```
a[0]=12;
```

```
a[9]=1;
```

Asignación con lectura

```
printf("Dame valor");
```

```
scanf("%d", &a[0])
```

```
printf("Dame valor");
```

```
scanf("%d", &a[9])
```

- Inicialización/Asignación

```
int a[5]={1,2,3,4,5}
```

```
int b[ ]={32,56,1}
```

```
char z[ ]={'L','u','i','s'};
```

```
char cad[]={“Hola Mundo”};
```

```
/* contiene \0 al final*/
```

Asignación

```
int b[1000];
```

```
for(i=0;i<1000;i++)
```

```
b[i]=i+1;
```

```
for(i=0;i<1000;i++){
```

```
printf("Dar valor");
```

```
scanf(" %d\n ", &b[i])
```

```
}
```

Uso de datos guardados en los arreglos

```
int a[10];
```

```
int suma;
```

```
suma = 0
```

```
suma=a[0]+a[1]+a[2]+...+a[9];
```

```
for(i=0;i<1000;i++)
```

```
    suma=suma+a[i];
```

11	12	13	14	15	16	17	18	19	20
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

Ejercicio

- Mostrar las direcciones asignadas a un arreglo de caracteres

¿Por qué usar arreglos unidimensionales?

Problema

- Considerar que en una universidad se conocen las calificaciones de 50 alumnos . Se necesita saber cuántos de estos tienen calificación superior al promedio del grupo.
- ¿Cómo resolver este problema?

Una Solución

i, cont: **Enteras**
AC,PROM, C: **reales**

Inicio

AC=1

i=1

Mientras i<= 50 Repetir **hacer**

 Escribe “Ingrese calificación”

 Leer C

 AC=AC+C

 i=i+1

Fin Mientras

Prom=AC/50

cont=0

i=1

Mientras (i<=50) **hacer**

 Escribe “Ingrese calificación”

 Leer C

 Si C > prom entonces

 cont=cont+1

 Fin Si

 i=i+1

Fin Mientras

Escribe cont

FIN

Otra solución (Variables)

cont: Entera
AC,PROM, C1,...C50: reales

Inicio

Leer C1,C2,... C50

$AC = C1 + C2 + \dots + C50$

$PROM = AC / 50$

cont=0

Si $C1 > PROM$ entonces

cont=cont+1

Fin Si

.....

Si $C50 > PROM$ entonces

cont=cont+1

Fin Si

Escribir cont

Fin

Solución con arreglos

- ¿Cómo?

calif[50], suma, prom: reales
i, numEst: entero

Inicio

numEst=0 suma=0

Para i=0 hasta i=n-1 hacer

 Escribe "Introduce calif"

 Leer calif[i]

 suma=suma+calif[i]

 i=i+1

Fin Para

prom=suma/50

Para i=0 hasta i=n-1 hacer

 Si calif[i] > prom

 numEst=numEst+1

 Fin Si

Fin Para

Escribe numEst

Fin

Arreglos bidimensionales

- Vista Conceptual (Matriz)

a[0][0]	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2]	a[2][3]

Declaración Arreglos bidimensionales en C estáticos o contiguos

tipoDato Nombre[tamañoD1][tamañoD2];

tipoDato Nombre[n renglones][n columnas];

short int a[3][4];

a[0][0]	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2]	a[2][3]

Representación en memoria

```
short int a[3][4];
```

FF00		a[0][0]
FF01		
FF02		a[0][1]
FF03		
FF04		a[0][2]
FF05		
FF06		a[0][3]
FF07		
FF08		a[1][0]
FF09		
FF0A		a[1][1]
FF0B		
FF0C		a[1][2]
FF0D		
FF0E		a[1][3]
FF0F		
FF10		a[2][0]
FF11		
FF12		a[2][1]
FF13		
FF14		a[2][2]
FF15		
FF16		a[2][3]
FF17		

Uso

```
int a[3][4];
```

Asignación

```
a[0][0]=12;
```

```
a[2][2]=1;
```

Asignación con lectura

```
printf("Dame valor");
```

```
scanf("%d", &a[0][0])
```

```
printf("Dame valor");
```

```
scanf("%d", &a[2][2])
```

a[0][0] = 12	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2] = 1	a[2][3]

```
int a[2][3]={51,52,53,54,55,56}  
int a[2][3]={{51,52,53},  
             {54,55,56}};
```

a[0][0]=51	a[0][1]=52	a[0][2]=53
a[1][0]=54	a[1][1]=55	a[1][2]=56

```
Suma=a[0][0]+a[0][1]+a[0][2];  
Suma2=a[0][0]+a[1][0]
```

Acceso a todos los elementos

Recorrer todos los componentes del arreglo 2D

```
for (i=0; i < nrenglones; i++ )  
    for( j=0; j<ncolumnas; j++)  
        a[i][j]=i+j;
```

```
dato=0  
for (i=0; i < nrenglones; i++ )  
    for( j=0; j<ncolumnas; j++)  
        dato+=a[i][j];
```

i=0	a[0][0]=0	a[0][1]=1	a[0][2]=2	a[0][3]=3
i=1	a[1][0]=1	a[1][1]=2	a[1][2]=3	a[1][3]=4
i=2	a[2][0]=2	a[2][1]=3	a[2][2]=4	a[2][3]=5
	j=0	j=1	j=2	j=3

¿Qué valor tiene dato?